

Technical Design Document (TDD)

F1tenth PoliMI

The car: stack, car interface and deployment

Document type:	Technical Design Document (TDD)
Version:	0.01
Date:	October 25, 2026
Author:	F1tenth PoliMI team
Classification:	Confidential

This document contains confidential and proprietary information of Politecnico di Milano.
Unauthorized access, reproduction, or distribution is prohibited.

Document History

Version	Date	Author	Changes
0.01	2026-10-25	F1tenth PoliMI team	Initial draft from the car-interface design proposal and the runbooks

Document Approvals

Role	Name	Date	Signature
Technical Lead			
Project Manager			
Management			

Contents

1 Introduction . . . **3**
- 1.1 Purpose . . . 3
- 1.2 Scope . . . 3
- 1.3 References . . . 3
- 1.4 Status Conventions . . . 3

2 Architecture . . . **3**
- 2.1 Component View . . . 3
- 2.2 Components . . . 3
- 2.3 Runtime View . . . 5
  - 2.3.1 Command path . . . 5
  - 2.3.2 Sensor path . . . 5
- 2.4 Deployment View . . . 6
  - 2.4.1 Machines and images . . . 6
  - 2.4.2 The car . . . 6
  - 2.4.3 Repositories . . . 6
  - 2.4.4 Launch . . . 7
- 2.5 Design Decisions . . . 7

3 Data . . . **8**

4 Interfaces . . . **8**
- 4.1 The car interface . . . 9
- 4.2 Path following . . . 9

5 Security & Privacy . . . **9**

6 Requirements Traceability . . . **9**

7 Implementation . . . **10**
- 7.1 Technology Stack . . . 10
- 7.2 Plan . . . 11
- 7.3 Development Standards . . . 11

8 Testing . . . **11**
- 8.1 Strategy . . . 12
- 8.2 Acceptance Criteria . . . 12

9 Open Items . . . **12**

1 Introduction

1.1 Purpose

This document defines how the car's software is built: the layers of the stack, the car interface the stack talks to, how the stack is started, where the car's measured numbers live, and how it is deployed on the car and on a developer's machine.

1.2 Scope

- **In scope:** the stack on the car, the car interface (topics and frames), the car's hardware layer, the repositories, the Docker images, the development standards.
- **Out of scope:** the simulator's internals (the gym, the bridge, `polimi_sim`). Here the simulator appears only as the second implementation of the car interface (`hw:=sim`). It gets its own documents, `RASD_Gym` and `TDD_Gym` **[TBD]**.

1.3 References

- `RASD_F1tenth` — requirements refined here.
- Car-interface design proposal, 24 Oct 2026 (the source of sections 2 and 4).
- Simulation plan 01.4, migration to Jazzy, the JAX gym and the simulator in our fork.
- Runbooks in `docs/software/`: `setup.md`, `docker.md`, `car.md`, `git.md`, `github-access.md`.

1.4 Status Conventions

[Decided] agreed **[Proposed]** proposed, waiting for the team's confirmation **[V2]** later phase **[TBD]** open.

> **Today versus target.** The images run ROS 2 Humble today and the drivers come from `humble-polimi`. Jazzy is decided (DEC-01) and the migration is planned (plan 01.4), not done. This document describes the Jazzy target; where today differs, it says so.

2 Architecture

> **Key point.** One stack, two cars. The stack talks to the car only through a fixed set of ROS topics and TF frames, the **car interface**. On the car, `f1tenth_system`'s drivers and `orbbec_camera` serve it. In the simulator, our fork of `f1tenth_gym_ros` serves the same interface. The stack cannot tell which one it is running on, and neither fork depends on anything of ours. **[Proposed]**

2.1 Component View

Three layers. The logic and the common layer are one stack and run unchanged everywhere. The hardware layer is exactly one of two, picked by the launch argument `hw:=car` or `hw:=sim`.

The seam sits below the mux, at `/ackermann_cmd`, the topic the VESC reads. The RASD requires joystick takeover, the deadman and AEB to work on the car and in the simulator (FR-SIM-002), so the mux and the safety nodes belong to the common layer, not to the hardware.

2.2 Components

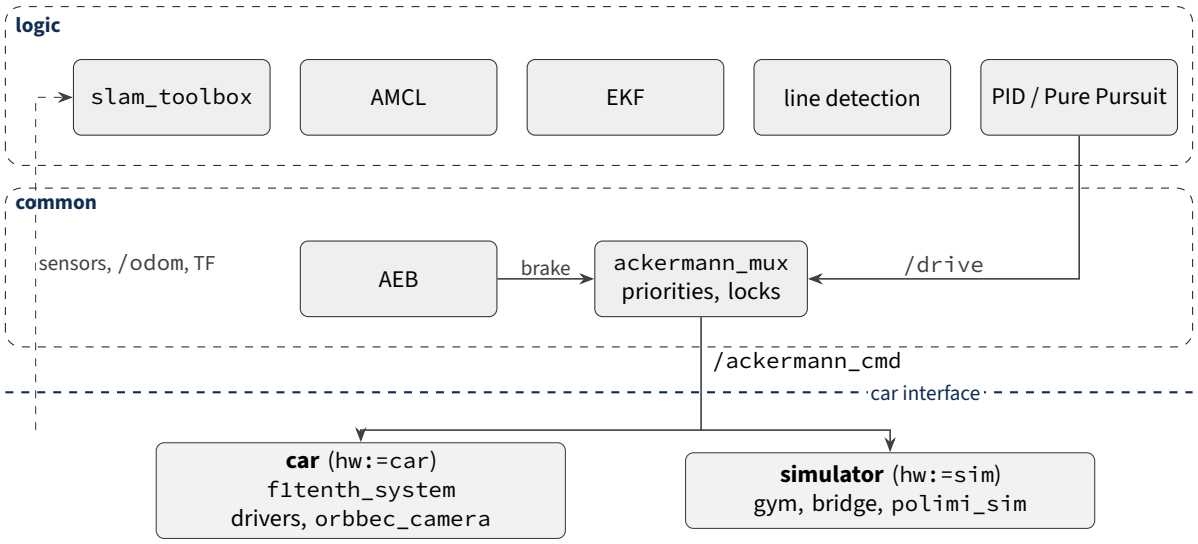


Figure 1: Layers. Only `/ackermann_cmd` goes down through the car interface; sensors, odometry and TF come up with the same names whichever hardware runs.

Component	Responsibility	Interfaces
Logic		
`slam_toolbox`	Map the room (FR-MAP)	`/scan`, TF → `/map`
AMCL	Pose in the room map (FR-LOC)	`/scan`, `/odom`, `/map` → TF `map`→`odom`
EKF (`robot_localization`)	Fused `odom` **[TBD: OI-08]**	`/odom`, IMU → TF `odom`→`base_link`
`polimi_lines`	Line detection (FR-LIN)	camera image, info → `nav_msgs/Path`
`polimi_control`	PID, Pure Pursuit (FR-PTH)	path, TF → `/drive`
Common		
`ackermann_mux`	Pick the highest-priority input still sending; locks (FR-SAF)	`/teleop`, `/drive`, `/joy` → `/ackermann_cmd`
`polimi_safety`	AEB (FR-SAF-004)	`/scan`, `/odom` → mux input
`polimi_bringup`	The car interface (README), launches, our configs	`stack.launch.py hw:=car|sim`
Hardware, car		
`vesc_driver`, `vesc_ackermann`	Motor, servo, odometry (FR-DRV-001)	`/ackermann_cmd` → VESC; VESC → `/odom`
`joy`, `joy_teleop`	Joystick (FR-DRV-002)	`/joy`, `/teleop`
`orbbec_camera`	SL450 LiDAR, Gemini 335L camera and IMU (FR-DRV-003)	`/scan`, image, info, IMU
Hardware, simulator (see `TDD_Gym`)		
fork of `f1tenth_gym_ros`	Serves the same interface	as the car

2.3 Runtime View

2.3.1 Command path

Autonomy publishes `/drive`. A person drives through `/teleop`, from the joystick on the car or the keyboard in the simulator. The mux picks by priority (joystick 100, autonomy 10, as in today's `mux.yaml`); the safety locks of OI-09 sit in the same mux. Below the seam, the car turns `/ackermann_cmd` into VESC commands; the simulator turns it into gym actions through its actuation model.

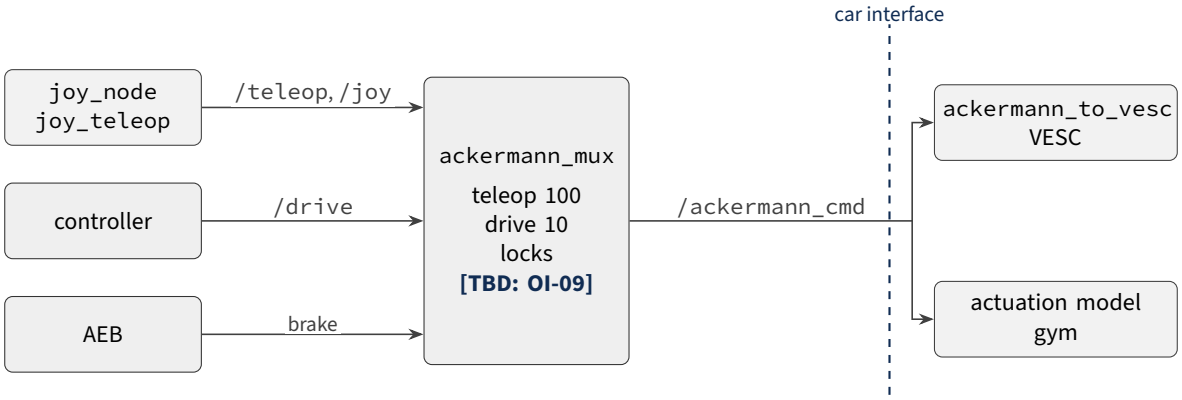


Figure 2: Command path. Grey boxes are hardware-specific: in the simulator a keyboard bridge publishes `/teleop` in place of the joystick nodes, and the car's VESC is replaced by the actuation model. Everything else is identical on car and simulator, so takeover, deadman and AEB are tested in the simulator as they run on the car.

2.3.2 Sensor path

Each topic in the middle column is part of the car interface. The car and the simulator each have one producer per topic; the stack subscribes only to the middle column. The simulator's ground truth stays out of it: it is for debugging and scoring only, never read by the stack (FR-SIM-003).

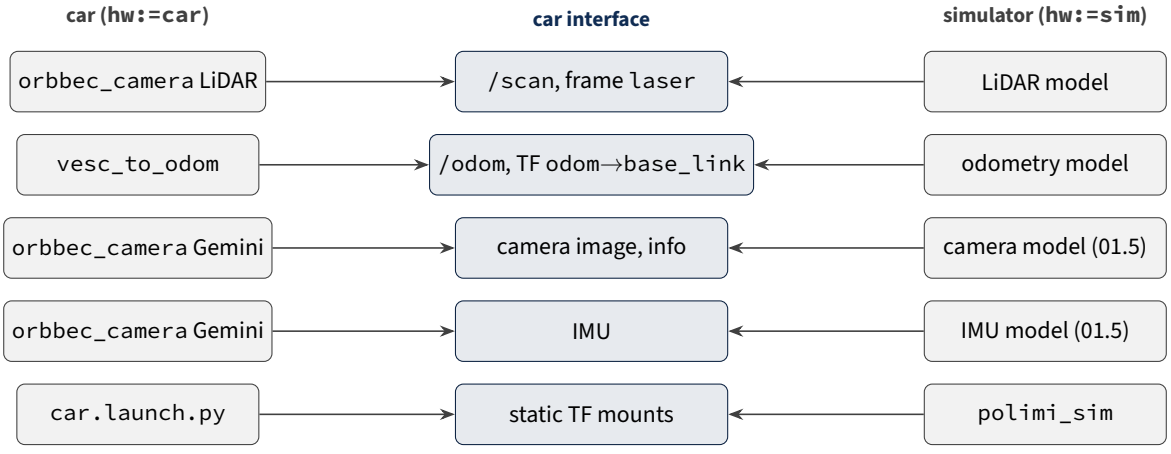


Figure 3: Sensor path. Same topic, same type, same frame, two producers. The mount TF comes from the same measured files on both sides (DEC-09), so the simulator places the LiDAR where the car's TF says it is.

2.4 Deployment View

2.4.1 Machines and images

One Docker image per machine type, built from `docker/ros/Dockerfile`. The repo's code, configs and maps are bind-mounted, so code is edited on the host and run in the container; `build/`, `install/` and `log/` live in Docker volumes.

Service	Machine	Simulator	Networking
`arm`	Apple Silicon Mac	yes	port 8765 (Foxglove), bound to 127.0.0.1
`x86`	Linux or Windows PC	yes	port 8765 (Foxglove), bound to 127.0.0.1
`jet`	Jetson Orin Nano on the car	no	host
`novnc`	optional, RViz in the browser	–	port 8080

Base image `ros:jazzy` on Ubuntu 24.04 (DEC-01); today `ros:humble`. Middleware `rmw_zenoh_cpp`; the Zenoh router starts with the container.

2.4.2 The car

Device	Link to the Orin Nano	Driver
Orbbec Pulsar SL450	Ethernet (the only port)	`orbbec_camera`, `lidar.launch.py`
Orbbec Gemini 335L	USB 3	`orbbec_camera`, `gemini_330_series.launch.py`
VESC 6 MkVI	USB, `/dev/sensors/vesc` (udev rule **[TBD]**)	`vesc_driver`
8BitDo Ultimate 2C	wireless dongle, USB	`joy`

2.4.3 Repositories

Repo (`tfc-org`)	Submodule path	Holds
`f1tenth_polimi`	– (this repo)	Our packages, configs, maps, Docker, docs
`f1tenth_system_polimi`	`ros2_ws/src/f1tenth_system`	Upstream drivers, branch `jazzy-polimi` (today `humble-polimi`)
`f1tenth_gym_polimi`	`ros2_ws/src/f1tenth_gym_ros` (plan 01.4)	Upstream bridge plus `polimi_sim`, branch `dev-jazzy-polimi`

Dependencies point one way only: from our repo outwards. Neither fork knows our stack exists (DEC-08).

2.4.4 Launch

One entry point, `stack.launch.py`, with `hw:=car` or `hw:=sim`. It always starts the common layer and the logic asked for, plus exactly one hardware launch. **[Proposed]**

- `stack.launch.py hw:=car|sim`
  - `common.launch.py`: `ackermann_mux` with our `mux.yaml`, safety.
  - `car.launch.py` (`hw:=car`): `f1tenth_system`'s driver nodes one by one, `orbbec_camera`, the mount TF from `config/car/`.
  - `sim.launch.py` (`hw:=sim`): includes the fork's `sim_car.launch.py` with `teleop_topic:=teleop` and `config_dir:=~/ws/config`, nothing else.
  - the logic: SLAM or localization, line detection, controller, as asked.

Today the car starts with `f1tenth_stack`'s `no_lidar_bringup_launch.py` and the simulator with `simcar`, which becomes a shortcut for `hw:=sim`.

2.5 Design Decisions

ID	Decision	Rationale / alternatives
DEC-01	ROS 2 Jazzy on every image **[Decided]**	Humble ends on 31 May 2027, before or during the summer school; Jazzy is supported to 31 May 2029. Lyrical is too new: no `f1tenth_gym_ros` branch, no JetPack for its Ubuntu. (OI-13, 21 Oct 2026.)
DEC-02	One image rebuild adds `nav2_amcl`, `robot_localization`, `image_transport_plugins`, `cv_bridge` **[Decided]**	In the Jazzy base apt layer, so AMCL and the EKF can be tried before any stack package exists. Python nodes never import `cv_bridge` (it breaks under the gym's NumPy 2) and convert images with NumPy. (OI-10, 22 Oct 2026.)
DEC-03	Seam below the mux, at `/ackermann_cmd` **[Proposed]**	The simulator replaces the VESC, the motor, the chassis and the sensors, nothing above them. With the mux above the seam, takeover, deadman and AEB are the same nodes in both places. (R1)
DEC-04	Plain topics are the contract; no extension mechanism **[Proposed]**	Our launch includes the fork's launch with two arguments; one switch, `hw:=`. No new code in the fork, and the contract can be checked with `ros2 topic info` on both sides. (R2)
DEC-05	`polimi_bringup` holds the contract and the launches; one package per function for the logic **[Proposed]**	Functions have different dependencies (the camera stack versus none) and their own tests; one bringup package keeps how it all starts in one place. (R3)
DEC-06	Our own `car.launch.py`, not `f1tenth_stack`'s bringup **[Proposed]**	Upstream's `bringup_launch.py` starts the Hokuyo driver, its own mux and a hard-coded laser TF, none of which match our car. `f1tenth_system` stays a pure upstream fork: the scope's "our fork, extended with all our drivers" becomes "its driver packages, launched by our bringup". (R4)
DEC-07	Our own `mux.yaml`, in `polimi_bringup/config/` **[Proposed]**	Starts from today's values (joystick 100, autonomy 10). OI-09 adds the e-stop and deadman locks and the AEB input, and those are ours, not upstream's. (R5)

ID	Decision	Rationale / alternatives
DEC-08	The simulator fork runs from a bare clone **[Proposed]**	`polimi_sim` drops the mux node and its `ackermann_mux` and `f1tenth_stack` dependencies, and gains `teleop_topic` (default `ackermann_cmd`). Replaces decision 8 of plan 01.4, which made the fork depend on `f1tenth_system`. (R6)
DEC-09	The hardware side publishes the mount TF, from the same files **[Proposed]**	Car: `car.launch.py` from `config/car/*.yaml`. Simulator: `polimi_sim` from the same files via `config_dir`. One file, two readers. (R7)
DEC-10	`odom`→`base_link` has one owner at a time **[Proposed]**	The odometry publishes it until the EKF lands (OI-08); then the EKF does, and `publish_tf` goes off on car and simulator in the same change. Two publishers make the tree jump. (R8)
DEC-11	Everything runs in Docker, code bind-mounted **[Decided]**	Same images on Mac, PC and car; the host needs only Docker and git.
DEC-12	Forks stay upstream plus upstreamable fixes; our code in `polimi_*` packages **[Decided]**	Upstream fixes can be pulled; only fixes we cannot make from outside go into a fork.

3 Data

The measured car lives in `ros2_ws/config/`, written once and read by both the car and the simulator. The simulator fork's defaults only fill gaps and never hold our measurements; it logs where each value came from.

File	Readers	Holds
`config/car/laser_mount.yaml`	car bringup, simulator	`base_link`→`laser`: today f1tenth's defaults 0.27, 0, 0.11 m, [MEASURE] on the car
`config/car/camera_mount.yaml` **[TBD]**	car bringup, simulator	`base_link`→`camera_link`
`config/lidar/sl450.yaml`	simulator	SL450 model: 40 Hz, 270°, 0.2°, ranges 0.05–25 m, noise, dropout
`config/maps/*.yaml`	simulator	track maps for the simulator
`maps/`	SLAM, AMCL	saved room maps
`polimi_bringup/config/mux.yaml` **[Proposed]**	mux	priorities, timeouts (0.2 s), locks
`polimi_bringup/config/` slam, amcl, ekf **[Proposed]**	logic	node parameters
`vesc.yaml` (`f1tenth_stack`)	VESC driver	gains, servo offset, wheelbase

4 Interfaces

4.1 The car interface

This table is the contract the stack is built against; it becomes the README of `polimi_bringup` **[Proposed]**. All message types are standard ROS ones, so no interface package is needed. Exact camera and IMU topic names follow `orbbec_camera`'s defaults, so the car needs no remaps **[TBD: T-02]**.

Only `/ackermann_cmd` goes from the stack to the hardware; every other row comes from the hardware.

Topic / frame	Type	Car	Simulator
`/ackermann_cmd`	`AckermannDriveStamped`	`ackermann_to_vesc`	actuation model
`/teleop`	`AckermannDriveStamped`	`joy_teleop`	keyboard bridge
`/joy`	`Joy`	`joy_node`	stand-in **[TBD: T-01]**
`/scan`, frame `laser`	`LaserScan`	`orbbec_camera` LiDAR	LiDAR model
`/odom`	`Odometry`	`vesc_to_odom`	odometry model
TF `odom`→`base_link`	TF	`vesc_to_odom`	odometry model
camera image, info	`Image`, `CameraInfo`	`orbbec_camera`	camera model (plan 01.5)
IMU	`Imu`	`orbbec_camera`	IMU model (plan 01.5)
TF `base_link`→`laser`, `camera_link`	static TF	`car.launch.py`	`polimi_sim`

`/drive` (`AckermannDriveStamped`) is inside the stack, not part of the car interface: any controller writes it, and it enters the mux at priority 10.

4.2 Path following

- **Input path:** `nav_msgs/Path` on one topic, any frame; offline paths latched, online paths recomputed per camera frame (FR-PTH-002, 005).
- **Input pose:** TF lookup from the path's frame to `base_link` (FR-PTH-003).
- **Output:** `/drive`. A stale online path stops the car (FR-PTH-007).

5 Security & Privacy

- **Physical safety comes first:** the mux, its locks and AEB (FR-SAF) are the safety design; they run on the car and in the simulator unchanged (DEC-03).
- **Network:** on Mac and PC the container's ports bind to 127.0.0.1 only. The car's container uses host networking; ROS traffic on the car's network is not authenticated **[TBD]**.
- **Secrets and personal data:** none in the stack. Camera images may show people in the room; nothing is recorded by default.
- **Repository access:** protected branches and required reviews, as in the runbook `github-access.md`.

6 Requirements Traceability

Requirement	Component(s)	Test(s)
FR-DRV-001	vesc_driver, vesc_ackermann	bench: ros2 topic echo /odom while driving by joystick
FR-DRV-002	joy, joy_teleop	bench: /teleop only while the enable button is held
FR-DRV-003	orbbec_camera	bench: rates of /scan, image, IMU
FR-DRV-004	car.launch.py (DEC-09)	tf2_echo base_link laser matches laser_mount.yaml
FR-MAP-001, 002	slam_toolbox	AC-V1-01
FR-LOC-001, 002	AMCL	AC-V1-01
FR-LOC-003	EKF (DEC-10)	**[TBD: OI-08]**
FR-LIN-001, 002	polimi_lines	recorded camera data; AC-V1-02
FR-PTH-001 to 008	polimi_control	unit tests on the error and window search; AC-V1-02, 03
FR-SAF-001 to 005	ackermann_mux, mux.yaml, polimi_safety	AC-V1-04, in the simulator first
FR-SIM-001 to 003	car interface, hw:=sim	ros2 topic info on both sides; AC-V1-02
FR-DOC-001	runbooks, RASD, TDD	review by someone outside the team
FR-ADV-001 to 003	**[V2]** **[TBD: OI-01]**	AC-V2-A
FR-ADV-011 to 013	**[V2]** **[TBD: OI-01]**	AC-V2-B
NFR-POR-001	images, DEC-01, DEC-11	the same launch on arm, x86, jet
NFR-VAR-001, 002	logic	laps on more than one layout and room
NFR-VAR-003	polimi_lines	**[TBD: OI-06]**, once floor and tape are known
NFR-PER-001	logic, controller	**[TBD]**: no target yet
NFR-PER-002	orbbec_camera LiDAR	bench: ros2 topic hz /scan
NFR-PER-003	orbbec_camera camera	bench: ros2 topic hz on the image
NFR-PER-004	orbbec_camera, vesc_driver	**[TBD]**: method to measure timestamp error
NFR-PER-005	mux.yaml	stop sending /drive: the car stops within 0.2 s
NFR-SAF-001	mux, locks, AEB	AC-V1-04, in the simulator and on the car
NFR-SAF-002	test plan	AC-V1-04 passed on the car before autonomous laps
NFR-MNT-001, 002	development standards (DEC-12)	PR review
FR-OFF-001	**[TBD: OI-11]**	optional

7 Implementation

7.1 Technology Stack

Layer	Choice	Why
Compute	Jetson Orin Nano Super, 500 GB NVMe	BOM; fits the F1TENTH deck
OS, container	Ubuntu 24.04 images, Docker; JetPack 6 on the car **[TBD: T-10]**	DEC-01, DEC-11
Middleware	ROS 2 Jazzy, `rmw_zenoh_cpp`	DEC-01
Drivers	`f1tenth_system` (`vesc`, `ackermann_mux`, `teleop_tools`), `orbbec_camera`	C-022
Mapping, localization	`slam_toolbox`, `nav2_amcl`, `robot_localization`	scope; DEC-02
Visualization	Foxglove bridge; RViz through noVNC	runs without a display on the host
Python	system Python of the image; `uv` on the host	house rule

7.2 Plan

1. **Plan 01.4:** Jazzy images, the JAX gym, the simulator fork as a submodule.
2. **`polimi_bringup`:** the car interface README, `stack.launch.py`, `sim.launch.py`, `common.launch.py`, our `mux.yaml`; `simcar` calls `hw:=sim`.
3. **Safety:** OI-09 locks and AEB, tested in the simulator.
4. **Logic:** mapping and localization, line detection, path following, as the base platform needs.
5. **Car bringup**, when the car is built: `car.launch.py`, udev rules, measured configs, safety tests on the car.
6. **Advanced topic** [V2], once OI-01 is decided.

7.3 Development Standards

- **Branches and reviews:** every change through a PR. Protected branches (`main`; the forks' `humble-*`, `jazzy-*`, `dev-humble*`, `dev-jazzy*`) block force pushes and deletion and need one approval.
- **Submodules:** changing a fork takes a commit there, then a pointer commit here; `push.recurseSubmodules=check` blocks a pointer to an unpushed commit. `scripts/setup_git.sh` sets this up and adds the `upstream` remotes.
- **Upstream sync:** merge upstream into our branch, never rewrite it; preview with `git log ours..upstream`.
- **Pins:** gym and bridge refs are bumped on purpose, never by following a branch (NFR-MNT-002).
- **Rebuilds:** code and configs need only `cb`; a `package.xml`, the Dockerfile or a pin needs an image rebuild.
- **Python:** `uv` on the host, never `pip install` directly.

8 Testing

8.1 Strategy

Level	Scope	Tooling / gate
Unit	Controller math, line fitting, AEB time to collision	`colcon test`, pytest
Contract	The car interface on both sides	`ros2 topic info` and `tf2_echo`: same names, types, frames on `hw:=car` and `hw:=sim`
System, simulator	Full flows: map, localize, laps, safety	AC-V1-01 to 04 in the simulator first
System, car	The same flows on the car	AC-V1-01 to 04 on the car; safety before autonomous laps (NFR-SAF-002)

8.2 Acceptance Criteria

As in `RASD_F1tenth`: AC-V1-01 to 04 for the base platform, AC-V2-A or AC-V2-B for the advanced topic. Leaving the lines is judged by the people watching on the car, and measured in the simulator.

9 Open Items

ID	Topic	Notes
T-01	Deadman and e-stop in the simulator	Docker Desktop on macOS has no USB passthrough, so there is no joystick in the container. The locks need a stand-in for `/joy`: a keyboard key that toggles the deadman, or a joystick read on the Mac and bridged in. Settle with OI-09.
T-02	Camera and IMU topic names [VERIFY]	From `orbbec_camera`'s Gemini launch, once the camera is on the bench; the simulator's camera model uses the same names.
T-03	LiDAR frame name [VERIFY]	From `orbbec_camera`'s `lidar.launch.py`; set it to `laser` in `car.launch.py` if it differs.
T-04	Confirm DEC-03 to DEC-10	The proposal amends plan 01.4: decision 8 withdrawn, M2 and M3 change, the scope's "Drivers" line is reworded.
T-05	Wheelbase	`vesc.yaml` has 0.25 m, the simulated odometry the gym's 0.3302 m. Measure the car and align both.
T-06	Mount measurements [MEASURE]	`laser_mount.yaml` holds f1tenth's defaults; the camera mount file does not exist yet.
T-07	VESC device	udev rule on the Jetson for `/dev/sensors/vesc`, expected by `vesc.yaml`.
T-08	`orbbec_camera` version	2.9.3 is in the Jazzy apt repo; check it with the SL450's firmware (plan 01.4, M-O8).
T-10	JetPack and Ubuntu 24.04 [VERIFY]	JetPack 6 is based on Ubuntu 22.04. Check that the `jet` service runs a `ros:jazzy` (24.04) container with `runtime: nvidia` on the Orin Nano.

ID	Topic	Notes
T-09	Unmatched remap	Today's bringup remaps `ackermann_cmd_out`→`ackermann_drive`, which matches nothing: the mux publishes `ackermann_cmd`. Gone with DEC-06.